

Repository Push & Change Process

nextvillage / Al'ing & Vibing platform

Documents the change-review process put in place July 2026, once the team grew beyond a single contributor.

1. Why This Changed

Until now, changes went straight to the `main` branch — whoever was working could push directly, with nothing checking that the change actually built or worked before it reached the live site. That gap already caused one real incident: a core signup feature was accidentally disabled by an unrelated commit and stayed broken in production for six weeks before anyone noticed.

Now that Silas and Divinegift are also making changes, a single unreviewed push is one person's mistake away from breaking the site for everyone. The process below adds an automatic safety check without adding much friction.

2. What Changed, In One Sentence

Changes to `main` now go through a **Pull Request** (PR) instead of a direct push, and GitHub automatically runs a check — **typecheck + build** — before the PR can be merged.

3. The Rules

Rule	Applies to
Direct pushes to <code>main</code> are blocked	Everyone except the repo owner (Kevin)
A Pull Request is required to merge into <code>main</code>	Everyone except the repo owner
The typecheck-and-build check must pass before merging	Everyone, including PRs opened by the owner
No approval/review from another person is currently required	— (kept intentionally light-weight for now)

Kevin can still push directly to `main` in a genuine emergency (repo owner/admin bypass), but this should be the exception, not the norm — even Kevin should default to using a PR.

4. What the Automatic Check Actually Does

Every PR (and every push to `main`) triggers a GitHub Actions workflow defined in `.github/workflows/ci.yml`. It:

1. Installs dependencies (`npm ci`)

2. **Typechecks** the whole codebase (`tsc --noEmit`) — catches type errors, broken imports, and typos that would otherwise only surface at runtime
3. **Builds** the production bundle (`npm run build`) — catches anything that compiles but doesn't actually build (missing files, bad imports, config errors)

It does *not* run automated tests (there aren't any meaningful ones yet) and does *not* check that a feature actually works in the browser — it only guarantees the code compiles and builds. Manual verification in the browser is still the contributor's responsibility for anything user-facing.

5. Step-by-Step: Making a Change

1 Create a branch for your change

```
git checkout development
git pull origin development
git checkout -b your-name/short-description-of-change
```

Example: `git checkout -b silas/fix-dashboard-timeout`

2 Make your changes and commit

```
git add .
git commit -m "Describe what changed and why"
```

3 Push your branch

```
git push -u origin your-name/short-description-of-change
```

4 Open a Pull Request

GitHub will show a banner with a "Compare & pull request" button right after you push — click it. Or go to the repository on GitHub.com and click **Pull requests** → **New pull request**. Set the base branch to `main` (or `development`, if that's the intended target — see Section 6) and your branch as the compare branch.

5 Wait for the check

Within about a minute, GitHub will show either ✓ All checks have passed or

✗ Some checks were not successful on the PR page.

- **If it passes:** click **Merge pull request**.
- **If it fails:** click "Details" next to the failed check to see the error output, fix it on your branch, commit, and push again — the check re-runs automatically.

6. Branch Strategy

Branch	Purpose
<code>main</code>	Production — what's actually deployed and live for users. Protected as described above.
<code>development</code>	Working/staging branch. Currently kept in sync with <code>main</code> ; feature branches are typically cut from here.

7. Team & Access

Person	GitHub account	Can bypass PR requirement?
Kevin Hallinan	<code>khallinan12345</code> (owner/admin)	Yes, but shouldn't by default
Silas	<code>girls-aing</code>	No
Divinegift	Not yet on GitHub	N/A — to be added once she creates an account

8. Quick Reference

Before you push: Is this going to `main` ? If yes, it needs a PR — you can't push directly.

PR failing? Click "Details" on the failed check, read the error, fix, commit, push again.

Not sure if your change is safe? Open the PR anyway — the check will catch build/type errors automatically, and Kevin can review before merging if needed.